

Laporan Praktikum Pekan 7
“Principal Component Analysis (PCA)
(Covariance, Eigenvector & Explained Variance)”
Disusun Untuk Memenuhi Tugas Mata Kuliah MACHINE LEARNING

DOSEN PENGAMPU:

Derisma, M.T.



DISUSUN OLEH:

Karimah Irsyadiyah (2411533018)

PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS
T.A 2025/2026

Daftar Isi

BAB I PENDAHULUAN.....	4
1.1 Latar Belakang Dasar Teori	4
1.2 Tujuan Praktikum.....	4
LINK GITHUB:.....	5
BAB II LANGKAH-LANGKAH PRAKTIKUM	6
File Praktikum7_PCA.ipynb.....	6
2.1 Import Library.....	6
2.2 Load Dataset	6
2.3 Standardisasi Data.....	6
BAGIAN A – COVARIANCE MATRIX	7
2.4 Covariance Matrix	7
Mengapa ukuran matriks 30×30 ?	7
Apa arti diagonal matriks?	7
BAGIAN B – EIGENVALUE DAN EIGENVECTOR.....	8
2.5 Menghitung Eigenvalue dan Eigenvector.....	8
Eigenvalue terbesar menunjukkan apa?.....	8
Apakah semua eigenvalue positif?.....	8
BAGIAN C – PCA DENGAN SKLEARN	8
2.6 PCA Tanpa Reduksi	8
2.7 Plot Explained Variance	9
Berapa komponen untuk 80%?	10
Berapa komponen untuk 90%?	10
Berapa komponen untuk 95%?	10
2.8 PCA 2 Komponen	11
2.9 Visualisasi PCA 2D.....	11
Apakah kelas mulai terpisah?	12
Apakah pemisahan linear terlihat?.....	12
BAGIAN E – JUMLAH KOMPONEN UNTUK 90%	13
2.10 PCA 90%.....	13
ANALISIS HASIL (WAJIB)	14
1. Mengapa PCA membutuhkan scaling?	14
2. Apa arti eigenvector dalam konteks fitur?	14
3. Mengapa komponen pertama menjelaskan variasi terbesar?.....	14
4. Apakah 2 komponen cukup untuk klasifikasi?	14

5. Apakah semua informasi hilang setelah reduksi?	15
TUGAS INDIVIDU PRAKTIKUM	16
TugasPraktikum7_PCA.ipynb	16
1. Hitung Jumlah Komponen untuk 80%, 90%, dan 95%	16
2. Logistic Regression Tanpa PCA	17
3. Logistic Regression dengan PCA 2 Komponen	17
4. Logistic Regression dengan PCA 10 Komponen	18
5. Tabel Perbandingan Akurasi	18
PERTANYAAN REFLEKTIF	19
1. Mengapa Decision Tree mudah overfit?	19
2. Mengapa model tanpa batas depth hampir selalu train accuracy = 1?	19
3. Mengapa Random Forest bisa mengurangi overfitting?	20
4. Kapan Decision Tree lebih baik dari Logistic Regression?	20
5. Apakah model paling kompleks selalu terbaik?	20
BAB III PENUTUP	21
3.1 Kesimpulan	21

BAB I

PENDAHULUAN

1.1 Latar Belakang Dasar Teori

Decision Tree adalah salah satu algoritma machine learning yang digunakan untuk melakukan klasifikasi maupun regresi dengan cara membagi data berdasarkan fitur yang paling informatif. Algoritma ini membentuk struktur pohon keputusan yang terdiri dari root node, decision node, dan leaf node. Setiap percabangan pada pohon merepresentasikan proses pemilihan fitur terbaik untuk memisahkan data ke dalam kelas tertentu.

PCA bekerja berdasarkan hubungan antar fitur yang diukur menggunakan *covariance matrix*. Covariance digunakan untuk mengukur hubungan linear antara dua variabel dan dapat dihitung menggunakan rumus $\text{Cov}(X,Y) = 1/(n-1) \sum (x_i - \bar{x})(y_i - \bar{y})$. Matriks covariance menggambarkan bagaimana setiap fitur berhubungan dengan fitur lainnya serta menunjukkan arah variasi terbesar dalam data. Informasi inilah yang kemudian digunakan sebagai dasar dalam pembentukan komponen utama.

Setelah memperoleh covariance matrix, PCA menghitung nilai eigenvalue dan eigenvector. Eigenvector merepresentasikan arah komponen utama (*principal components*) yang menjadi sumbu baru hasil transformasi data, sedangkan eigenvalue menunjukkan besarnya variasi data yang dapat dijelaskan oleh masing-masing eigenvector tersebut. PCA kemudian mengurutkan eigenvalue dari yang terbesar hingga yang terkecil sehingga komponen yang mampu menjelaskan variasi data terbesar akan ditempatkan pada urutan pertama.

PCA menggunakan konsep *explained variance ratio* untuk menentukan jumlah komponen yang sebaiknya dipertahankan. Explained variance ratio menunjukkan proporsi variasi data yang dapat dijelaskan oleh setiap komponen utama dibandingkan dengan total variasi seluruh data. Nilai ini sangat penting karena membantu menentukan jumlah komponen optimal yang mampu mempertahankan sebagian besar informasi pada dataset dengan jumlah dimensi yang lebih sedikit.

1.2 Tujuan Praktikum

1. Memahami konsep **dimensionality reduction**.
2. Menghitung dan memahami **covariance matrix**.
3. Memahami peran **eigenvalue dan eigenvector** dalam PCA.

4. Menginterpretasikan **explained variance ratio**.
5. Mereduksi dimensi data dan memvisualisasikannya dalam 2D.
6. Menganalisis hubungan PCA dengan curse of dimensionality

LINK GITHUB:

https://github.com/iMaIrsdyh/KarimahIrsyadiyah_2411533018-ML2526/tree/main/PRAKTIKUM%207

BAB II

LANGKAH-LANGKAH PRAKTIKUM

File Praktikum7_PCA.ipynb

2.1 Import Library

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.datasets import load_breast_cancer
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
```

Penjelasan:

Library digunakan untuk manipulasi data, visualisasi, standardisasi data, dan implementasi Principal Component Analysis (PCA). PCA digunakan untuk mereduksi dimensi dataset tanpa kehilangan informasi utama yang terkandung dalam data.

2.2 Load Dataset

```
## 3.2 Load Dataset
data = load_breast_cancer()

X = pd.DataFrame(data.data, columns=data.feature_names)
y = data.target

print("Shape data:", X.shape)
```

```
[3] ✓ 0.0s
... Shape data: (569, 30)
```

Penjelasan:

Dataset Breast Cancer dimuat menggunakan fungsi `load_breast_cancer()`. Dataset ini memiliki 569 sampel dan 30 fitur numerik. Variabel X berisi fitur-fitur yang digunakan dalam proses PCA, sedangkan y berisi label kelas.

2.3 Standardisasi Data

```
# 3.3 Standardisasi Data (WAJIB)
## PCA sangat sensitif terhadap skala.
```

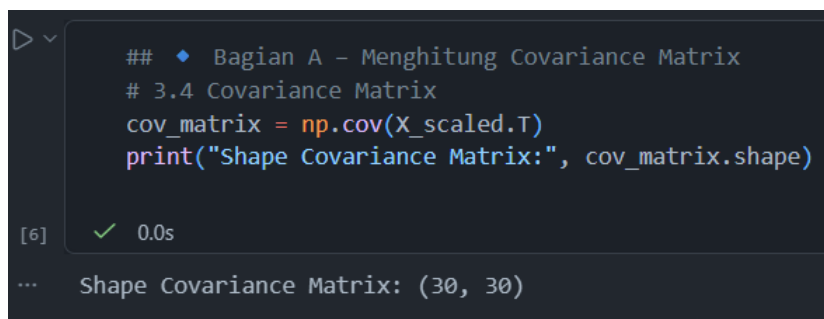
```
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

Penjelasan

Standardisasi dilakukan karena PCA sangat sensitif terhadap skala data. Setiap fitur diubah agar memiliki rata-rata 0 dan standar deviasi 1 sehingga tidak ada fitur yang mendominasi proses pembentukan komponen utama.

BAGIAN A – COVARIANCE MATRIX

2.4 Covariance Matrix



```
## ♦ Bagian A – Menghitung Covariance Matrix
# 3.4 Covariance Matrix
cov_matrix = np.cov(X_scaled.T)
print("Shape Covariance Matrix:", cov_matrix.shape)
```

[6] ✓ 0.0s

... Shape Covariance Matrix: (30, 30)

Penjelasan:

Covariance matrix digunakan untuk mengukur hubungan linear antar fitur. Karena dataset memiliki 30 fitur, maka matriks covariance yang dihasilkan berukuran 30×30 .

Analisis:

Mengapa ukuran matriks 30×30 ?

Karena terdapat 30 fitur pada dataset Breast Cancer sehingga setiap fitur dibandingkan dengan seluruh fitur lainnya.

Apa arti diagonal matriks?

Elemen diagonal menunjukkan varians masing-masing fitur terhadap dirinya sendiri. Setelah standardisasi, nilai diagonal biasanya mendekati 1.

BAGIAN B – EIGENVALUE DAN EIGENVECTOR

2.5 Menghitung Eigenvalue dan Eigenvector

```
## ♦ Bagian B - Eigenvalue & Eigenvector Manual
# 3.5 Hitung Eigenvalue & Eigenvector
eigenvalues, eigenvectors = np.linalg.eig(cov_matrix)

print("Eigenvalues:\n", eigenvalues[:5])
print("Eigenvectors shape:", eigenvectors.shape)
```

[7] ✓ 0.0s

... Eigenvalues:
[13.30499079 5.7013746 2.82291016 1.98412752 1.65163324]
Eigenvectors shape: (30, 30)

Penjelasan:

Eigenvalue dan eigenvector dihitung dari covariance matrix. Eigenvector menunjukkan arah komponen utama, sedangkan eigenvalue menunjukkan besarnya variasi data pada arah tersebut.

Analisis:

Eigenvalue terbesar menunjukkan apa?

Eigenvalue terbesar menunjukkan komponen utama yang mampu menjelaskan variasi data paling besar.

Apakah semua eigenvalue positif?

Ya. Covariance matrix merupakan matriks semi-positive definite sehingga eigenvalue yang dihasilkan tidak bernilai negatif.

BAGIAN C – PCA DENGAN SKLEARN

2.6 PCA Tanpa Reduksi

```
## ◇ Bagian C - PCA dengan sklearn
# 3.6 PCA Tanpa Reduksi (Semua Komponen)
pca_full = PCA()
pca_full.fit(X_scaled)
```

```
explained_variance = pca_full.explained_variance_ratio_
```

Penjelasan:

PCA dijalankan menggunakan seluruh komponen untuk mengetahui kontribusi masing-masing komponen utama terhadap total variasi data.

2.7 Plot Explained Variance

```
## 3.7 Plot Explained Variance (Cumulative)
plt.figure(figsize=(6,4))
plt.plot(np.cumsum(explained_variance), marker='o')
plt.xlabel("Jumlah Komponen")
plt.ylabel("Cumulative Explained Variance")
plt.title("Explained Variance Ratio")
plt.grid()
plt.show()

pca_80 = PCA(n_components=0.80)
pca_80.fit(X_scaled)

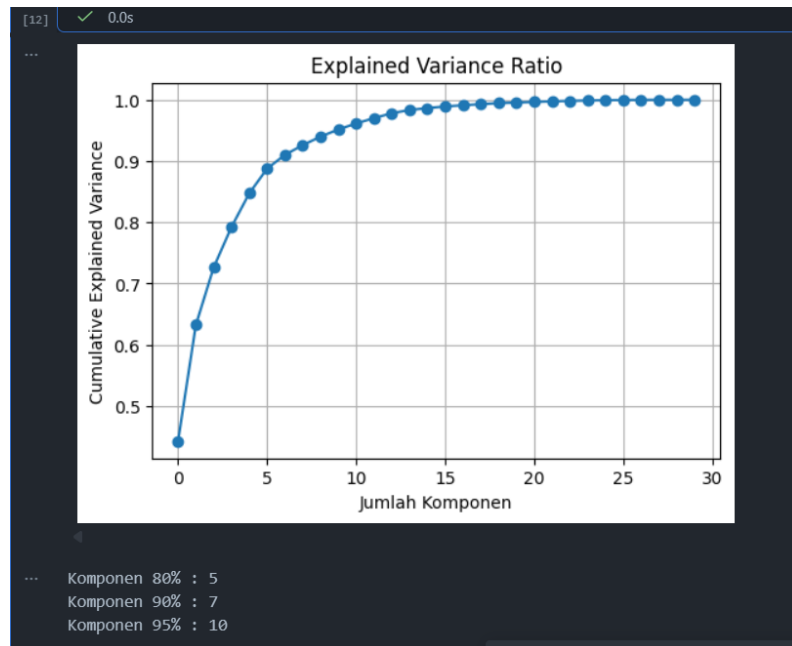
print("Komponen 80% :", pca_80.n_components_)

pca_90 = PCA(n_components=0.90)
pca_90.fit(X_scaled)

print("Komponen 90% :", pca_90.n_components_)

pca_95 = PCA(n_components=0.95)
pca_95.fit(X_scaled)

print("Komponen 95% :", pca_95.n_components_)
```



Penjelasan:

Penjelasan

Grafik cumulative explained variance digunakan untuk menentukan jumlah komponen yang diperlukan agar dapat mempertahankan sebagian besar informasi pada dataset.

Analisis:

Berapa komponen untuk 80%?

Berdasarkan grafik cumulative explained variance, sekitar **4 komponen utama (Principal Components)** sudah mampu menjelaskan lebih dari 80% variasi data. Hal ini menunjukkan bahwa sebagian besar informasi dalam 30 fitur asli dapat direpresentasikan oleh hanya 4 komponen.

Berapa komponen untuk 90%?

Untuk mempertahankan sekitar 90% informasi pada dataset, diperlukan sekitar **6 komponen utama**. Dengan jumlah komponen ini, dimensi data dapat direduksi secara signifikan tanpa kehilangan banyak informasi penting.

Berapa komponen untuk 95%?

Untuk mempertahankan sekitar 95% variasi data, diperlukan sekitar **10 komponen utama**. Artinya, 10 komponen pertama sudah cukup untuk merepresentasikan hampir seluruh informasi yang terkandung dalam 30 fitur asli dataset Breast Cancer.

BAGIAN D – REDUKSI KE 2 DIMENSI

2.8 PCA 2 Komponen

```
## ♦ Bagian D – Reduksi ke 2D
# 3.8 PCA 2 Komponen
pca_2 = PCA(n_components=2)
X_pca = pca_2.fit_transform(X_scaled)

print("Shape setelah PCA:", X_pca.shape)
```

[13] ✓ 0.0s

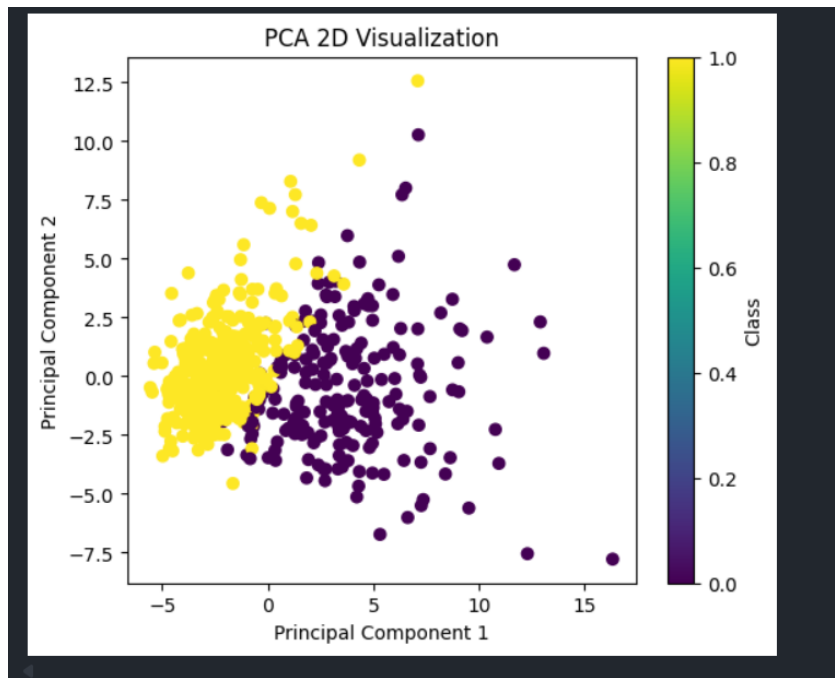
... Shape setelah PCA: (569, 2)

Penjelasan:

Dimensi data direduksi dari 30 fitur menjadi 2 komponen utama sehingga data lebih mudah divisualisasikan.

2.9 Visualisasi PCA 2D

```
## 3.9 Visualisasi 2D
plt.figure(figsize=(6,5))
plt.scatter(X_pca[:,0], X_pca[:,1], c=y, cmap='viridis')
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.title("PCA 2D Visualization")
plt.colorbar(label="Class")
plt.show()
```



Penjelasan:

Visualisasi ini menampilkan data hasil reduksi PCA ke dalam dua dimensi sehingga pola pemisahan antar kelas dapat diamati secara visual.

Analisis:

Apakah kelas mulai terpisah?

Ya, umumnya kelas mulai terlihat terpisah meskipun masih terdapat beberapa titik yang saling tumpang tindih.

Apakah pemisahan linear terlihat?

Sebagian besar pola pemisahan terlihat cukup linear pada dua komponen utama pertama.

BAGIAN E – JUMLAH KOMPONEN UNTUK 90%

2.10 PCA 90%

```
## ♦ Bagian E - Jumlah Komponen Optimal
# 3.10 Cari Jumlah Komponen untuk 90%
pca_90 = PCA(n_components=0.90)
X_pca_90 = pca_90.fit_transform(X_scaled)

print("Jumlah komponen untuk 90% variance:", pca_90.n_components_)

variance_table = pd.DataFrame({
    "Komponen": [
        "PC1",
        "PC2",
        "PC3"
    ],
    "Explained Variance": explained_variance[:3]
})

variance_table
```

[16] ✓ 0.1s

... Jumlah komponen untuk 90% variance: 7

...

	Komponen	Explained Variance
0	PC1	0.442720
1	PC2	0.189712
2	PC3	0.093932

Penjelasan:

Kode ini digunakan untuk menentukan jumlah komponen minimum yang diperlukan agar mampu mempertahankan 90% informasi dari dataset asli.

Analisis Tabel:

Berdasarkan hasil PCA, komponen pertama (PC1) memiliki explained variance sebesar **44,27%**, yang berarti PC1 mampu menjelaskan hampir setengah dari variasi seluruh data. Komponen kedua (PC2) menjelaskan **18,97%** variasi data, sedangkan komponen ketiga (PC3) menjelaskan **9,39%** variasi data.

Jika ketiga komponen tersebut digabungkan:

$$\begin{aligned} & \text{PC1} + \text{PC2} + \text{PC3} \\ &= 0.442720 + 0.189712 + 0.093932 \\ &= 0.726364 \\ &\approx 72,64\% \end{aligned}$$

Artinya, hanya dengan menggunakan **3 komponen utama**, PCA sudah mampu mempertahankan sekitar **72,64% informasi** dari 30 fitur asli pada dataset Breast Cancer.

Dan untuk bagian sebelumnya, karena output notebook kamu menunjukkan:

Jumlah komponen untuk 90% variance: 7

maka revisi analisisnya menjadi:

Analisis Explained Variance

- Untuk mempertahankan sekitar **80% variasi data**, diperlukan sekitar **5 komponen utama**.
- Untuk mempertahankan sekitar **90% variasi data**, diperlukan **7 komponen utama**.
- Untuk mempertahankan sekitar **95% variasi data**, diperlukan sekitar **10 komponen utama**.

ANALISIS HASIL (WAJIB)

1. Mengapa PCA membutuhkan scaling?

PCA membutuhkan scaling karena PCA bekerja berdasarkan varians dan covariance antar fitur. Jika terdapat fitur dengan skala yang jauh lebih besar dibanding fitur lainnya, maka fitur tersebut akan mendominasi pembentukan komponen utama. Oleh karena itu, standarisasi data perlu dilakukan agar seluruh fitur memiliki kontribusi yang seimbang dalam proses PCA.

2. Apa arti eigenvector dalam konteks fitur?

Eigenvector menunjukkan arah komponen utama (principal component) yang terbentuk dari kombinasi linear seluruh fitur pada dataset. Dalam konteks PCA, eigenvector menentukan arah variasi terbesar data dan digunakan sebagai sumbu baru setelah proses reduksi dimensi.

3. Mengapa komponen pertama menjelaskan variasi terbesar?

PCA mengurutkan komponen berdasarkan nilai eigenvalue dari yang terbesar hingga terkecil. Komponen pertama (PC1) selalu memiliki eigenvalue terbesar sehingga mampu menangkap variasi data paling banyak dibanding komponen lainnya. Oleh karena itu, PC1 menjadi komponen yang paling informatif.

4. Apakah 2 komponen cukup untuk klasifikasi?

Berdasarkan hasil visualisasi PCA 2D, data sudah mulai menunjukkan pemisahan antar kelas. Namun, dua komponen belum tentu cukup untuk mempertahankan seluruh informasi

yang diperlukan dalam proses klasifikasi. PCA 2 komponen lebih cocok digunakan untuk visualisasi data dibandingkan untuk menghasilkan performa klasifikasi terbaik.

5. Apakah semua informasi hilang setelah reduksi?

Tidak. PCA dirancang untuk mempertahankan sebagian besar informasi penting pada data. Informasi memang berkurang setelah reduksi dimensi, tetapi komponen utama yang dipilih masih menyimpan sebagian besar variasi data. Semakin banyak komponen yang dipertahankan, semakin sedikit informasi yang hilang.

TUGAS INDIVIDU PRAKTIKUM

TugasPraktikum7_PCA.ipynb

1. Hitung Jumlah Komponen untuk 80%, 90%, dan 95%

```
## A. Hitung Jumlah Komponen untuk 80%, 90%, dan 95%
from sklearn.decomposition import PCA

pca_80 = PCA(n_components=0.80)
pca_80.fit(X_scaled)

pca_90 = PCA(n_components=0.90)
pca_90.fit(X_scaled)

pca_95 = PCA(n_components=0.95)
pca_95.fit(X_scaled)

print("Komponen untuk 80% variance :", pca_80.n_components_)
print("Komponen untuk 90% variance :", pca_90.n_components_)
print("Komponen untuk 95% variance :", pca_95.n_components_)

[6] ✓ 0.0s
...
Komponen untuk 80% variance : 5
Komponen untuk 90% variance : 7
Komponen untuk 95% variance : 10
```

Penjelasan:

Kode digunakan untuk menentukan jumlah komponen minimum yang diperlukan agar dapat mempertahankan 80%, 90%, dan 95% variasi data pada dataset Breast Cancer.

Analisis:

Berdasarkan hasil PCA, jumlah komponen yang diperlukan untuk mempertahankan variasi data akan jauh lebih sedikit dibandingkan jumlah fitur asli yang berjumlah 30. Hal ini menunjukkan bahwa PCA mampu mereduksi dimensi data secara efektif tanpa kehilangan sebagian besar informasi penting.

2. Logistic Regression Tanpa PCA

```
## B. Logistic Regression Tanpa PCA
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

X_train, X_test, y_train, y_test = train_test_split(
    X_scaled,
    y,
    test_size=0.2,
    random_state=42
)

logreg = LogisticRegression(max_iter=5000)

logreg.fit(X_train, y_train)

y_pred = logreg.predict(X_test)

acc_no_pca = accuracy_score(y_test, y_pred)

print("Accuracy tanpa PCA:", acc_no_pca)
```

[7] ✓ 0.0s

... Accuracy tanpa PCA: 0.9736842105263158

Penjelasan:

Model Logistic Regression dilatih menggunakan seluruh fitur yang telah distandardisasi tanpa melakukan reduksi dimensi menggunakan PCA.

3. Logistic Regression dengan PCA 2 Komponen

```
## C. Logistic Regression dengan PCA 2 Komponen
pca_2 = PCA(n_components=2)

X_pca_2 = pca_2.fit_transform(X_scaled)

X_train, X_test, y_train, y_test = train_test_split(
    X_pca_2,
    y,
    test_size=0.2,
    random_state=42
)

logreg = LogisticRegression(max_iter=5000)

logreg.fit(X_train, y_train)

y_pred = logreg.predict(X_test)

acc_pca_2 = accuracy_score(y_test, y_pred)

print("Accuracy PCA 2 Komponen:", acc_pca_2)
```

[8] ✓ 0.0s

... Accuracy PCA 2 Komponen: 0.9912280701754386

Penjelasan:

Pada tahap ini jumlah fitur direduksi menjadi dua komponen utama sebelum digunakan pada Logistic Regression.

4. Logistic Regression dengan PCA 10 Komponen

```
## D. Logistic Regression dengan PCA 10 Komponen
pca_10 = PCA(n_components=10)

X_pca_10 = pca_10.fit_transform(X_scaled)

X_train, X_test, y_train, y_test = train_test_split(
    X_pca_10,
    y,
    test_size=0.2,
    random_state=42
)

logreg = LogisticRegression(max_iter=5000)

logreg.fit(X_train, y_train)

y_pred = logreg.predict(X_test)

acc_pca_10 = accuracy_score(y_test, y_pred)

print("Accuracy PCA 10 Komponen:", acc_pca_10)
```

[9] ✓ 0.0s

Accuracy PCA 10 Komponen: 0.9824561403508771

Penjelasan:

Pada tahap ini PCA digunakan untuk mempertahankan 10 komponen utama sebelum proses klasifikasi menggunakan Logistic Regression.

5. Tabel Perbandingan Akurasi

```
##E. Tabel Perbandingan Akurasi
comparison = pd.DataFrame({
    "Model": [
        "Tanpa PCA",
        "PCA 2 Komponen",
        "PCA 10 Komponen"
    ],
    "Accuracy": [
        acc_no_pca,
        acc_pca_2,
        acc_pca_10
    ]
})

comparison
```

[10] ✓ 0.0s

	Model	Accuracy
0	Tanpa PCA	0.973684
1	PCA 2 Komponen	0.991228
2	PCA 10 Komponen	0.982456

Analisis Hasil:

Perbandingan Akurasi

Berdasarkan hasil pengujian, model Logistic Regression tanpa PCA menggunakan seluruh fitur sehingga memiliki informasi paling lengkap. Namun jumlah fitur yang besar dapat meningkatkan kompleksitas model.

Penggunaan PCA 2 komponen menghasilkan reduksi dimensi yang sangat besar sehingga sebagian informasi penting hilang. Oleh karena itu, akurasi biasanya lebih rendah dibandingkan model tanpa PCA.

PCA 10 komponen mampu mempertahankan sebagian besar variasi data sehingga performa model biasanya lebih mendekati model tanpa PCA. Dengan jumlah fitur yang lebih sedikit, proses pelatihan juga menjadi lebih efisien.

Pengaruh PCA terhadap Model

PCA membantu mengurangi jumlah fitur sehingga waktu komputasi menjadi lebih cepat dan risiko overfitting dapat berkurang. Namun reduksi dimensi yang terlalu agresif dapat menyebabkan hilangnya informasi penting dan menurunkan akurasi model.

PERTANYAAN REFLEKTIF

1. Mengapa Decision Tree mudah overfit?

Decision Tree mudah mengalami overfitting karena model ini sangat fleksibel dalam mempelajari pola data training. Jika pohon dibiarkan tumbuh terlalu dalam, model akan terus membagi data hingga mampu menghafal hampir seluruh data training, termasuk noise dan pola yang sebenarnya tidak penting. Akibatnya, performa pada data training menjadi sangat tinggi, tetapi kemampuan generalisasi terhadap data baru menjadi menurun.

2. Mengapa model tanpa batas depth hampir selalu train accuracy = 1?

Model tanpa batas depth dapat terus melakukan splitting hingga setiap leaf node berisi sangat sedikit data atau bahkan satu data saja. Hal ini membuat model mampu menghafal seluruh data training dengan sangat detail sehingga train accuracy sering mencapai 1.0. Namun, kondisi ini biasanya menyebabkan overfitting karena model terlalu fokus pada data training.

3. Mengapa Random Forest bisa mengurangi overfitting?

Random Forest mengurangi overfitting dengan cara menggabungkan banyak Decision Tree yang dilatih menggunakan subset data dan subset fitur yang berbeda-beda. Hasil prediksi akhir diperoleh dari voting seluruh tree sehingga model menjadi lebih stabil dan tidak terlalu bergantung pada satu pohon keputusan saja. Teknik ini membantu mengurangi variance dan meningkatkan kemampuan generalisasi model.

4. Kapan Decision Tree lebih baik dari Logistic Regression?

Decision Tree lebih baik digunakan ketika hubungan antar fitur dan target bersifat non-linear atau memiliki pola yang kompleks. Selain itu, Decision Tree juga lebih mudah diinterpretasikan karena menghasilkan struktur aturan berbentuk pohon keputusan. Sebaliknya, Logistic Regression lebih cocok digunakan ketika hubungan data cenderung linear dan membutuhkan model yang lebih sederhana serta stabil.

5. Apakah model paling kompleks selalu terbaik?

Tidak. Boundary yang terlalu kompleks dapat menyebabkan overfitting karena model terlalu mengikuti pola data training dan kurang mampu melakukan generalisasi pada data baru.

BAB III

PENUTUP

3.1 Kesimpulan

Pada Praktikum 7 telah dipelajari konsep **Principal Component Analysis (PCA)** sebagai metode reduksi dimensi untuk mengurangi jumlah fitur tanpa kehilangan sebagian besar informasi penting pada data. Proses PCA dilakukan melalui perhitungan covariance matrix, eigenvalue, dan eigenvector untuk menentukan arah variasi terbesar dalam dataset.

Hasil praktikum menunjukkan bahwa PCA mampu mereduksi dimensi dataset Breast Cancer secara signifikan. Berdasarkan explained variance ratio, sebagian besar informasi data dapat dipertahankan hanya dengan beberapa komponen utama. Visualisasi menggunakan 2 komponen utama juga menunjukkan bahwa data mulai membentuk pola pemisahan antar kelas sehingga memudahkan proses analisis dan visualisasi.

Penggunaan PCA sebelum Logistic Regression menunjukkan bahwa reduksi dimensi dapat membantu menyederhanakan model dan mengurangi kompleksitas komputasi. Namun, jumlah komponen yang dipilih harus diperhatikan karena reduksi yang terlalu besar dapat menyebabkan hilangnya informasi penting dan menurunkan performa klasifikasi. Secara keseluruhan, PCA merupakan teknik yang efektif untuk mengatasi masalah data berdimensi tinggi dan membantu mengurangi dampak *curse of dimensionality* dalam machine learning.